

PIGGY

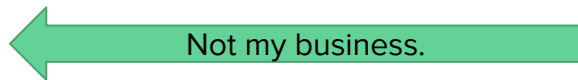
Precise, Interpretable Grammar-based Generator, yippee!

DELPH-IN 2026

What is PIGGY?

- PIGGY is designed to perform one of the steps in a data-to-text generation pipeline
- A pipeline for this task may involve, for each data point,

1. Select which data to verbalize as text

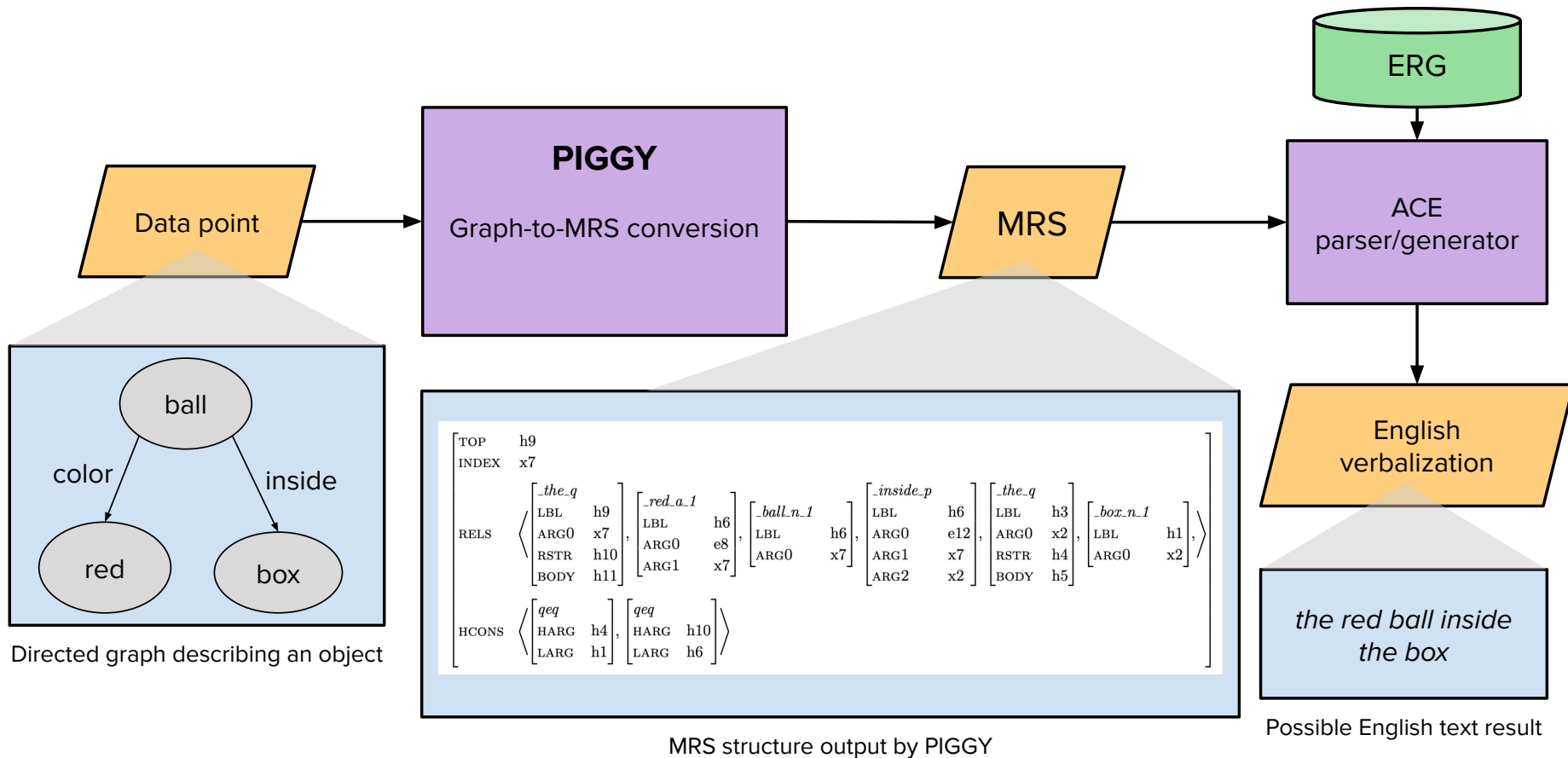


2. Build a semantic representation from that chosen data



3. Produce plain text as the final output given that s
representation





Components of PIGGY

- Fundamental Semantic Operations
- Semantic Composition Library (SCL)
- Lexicon Infrastructure
- Data-to-MRS algorithm

Fundamental Semantic Operations

Fundamental Semantic Operations

- Functions that either
 1. produce SEMENTs from scratch
 2. compose new SEMENTs using two input SEMENTs
- Inspired by the Semantic Algebra from Copestake et. al, 2001

SEMENT comparison

Copestake et. al, 2001

- HOOK — $\{[|b|, i, xarg]\}$
- SLOTS — $\emptyset$ or $\{[|b|', i', x']\}$
 - Named for syntactic(?) role, e.g. *subj*
- RELS — Bag of EPs, each containing:
 - A handle
 - A relation (e.g. `_cake_n_1` in the ERG)
 - Zero or more ordinary variable arguments
 - Zero or more handles, corresponding to scopal arguments
- EQs — Set of equalities between variables
- HCONS — Set of handle constraints

Conrad, TBD (2026 or so help me...)

- TOP — $|b|$
- INDEX — i
- SLOTS — $\emptyset$ or $\{[|b|' \mid i']\}$
 - i.e. EITHER handle or ordinary variable
 - named as bleached semantic argument, e.g. ARG1
- RELS — Bag of EPs, each containing:
 - A handle
 - A relation (e.g. `_cake_n_1` in the ERG)
 - Zero or more arguments (ordinary and scopal in same list)
- EQs — Set of equalities between variables
- HCONS — Set of handle constraints

Operations to create “lexical SEMENTs”

- `op_create_base_SEMENT` — create a lexical SEMENT with one EP given a predicate label
- `op_create_CARG_SEMENT` — create a lexical SEMENT with one EP that has a CARG slot given a predicate label and the value to insert for the CARG

Semantic Operation Comparison

Copestake et. al, 2001

For a slot labeled x:

- $\text{RES.HOOK} = \text{FUNC.HOOK}$
- $\text{RES.SLOTS} = (\text{FUNC.SLOTS} - \text{FUNC.SLOTS.x}) \oplus \text{ARG.SLOTS}$
- $\text{RES.RELS} = \text{FUNC.RELS} \oplus \text{ARG.RELS}$
- $\text{RES.EQS} = \text{Tr}(\text{FUNC.EQS} \cup \text{ARG.EQS} \cup \{\text{FUNC.SLOTS.x} = \text{ARG.HOOK}\})$
- $\text{RES.HCONS} = \text{FUNC.HCONS} \oplus \text{ARG.HCONS}$

Conrad, TBD (2026 or so help me...)

- Unchanged:
 - $\text{RES.RELS} = \text{FUNC.RELS} \oplus \text{ARG.RELS}^{**}$
 - $\text{RES.HCONS} = \text{FUNC.HCONS} \oplus \text{ARG.HCONS}$
- Changed:
 - Determination of HOOK
 - Determination of SLOTS
 - Determination of EQS
- Added:
 - Identify TOPs in non-scopal operations

*My semantic operations are not named for the slot they plug, so you do need to specify that information manually, but the result is still the sum of both lists minus the plugged slot

**Technically I may not be able to simply add the RELS lists and might have to “settify” them and do a union in control-type structures because I lack XARG

Setting the HOOK

- In Copestake et. al, 2001, the HOOK of the result SEMENT always comes from the functor SEMENT
- For the SEMENT to come out correctly with this setup, lexical entries(?) must be specified appropriately
 - e.g. an adjective identifies its own INDEX with its ARG1 rather than the ARG0
- Since I only have information from the SEMI, I default to identifying every lexical SEMENT's INDEX with the ARG0
- If I use the operation as defined in Copestake et. al, 2001, then the SEMENT for *tasty cookie* will have its INDEX identified with tasty.ARG0, which is not what we want

Setting the HOOK

- To handle this, I have two “kinds” of semantic operators:
 - Semantic operators where the result SEMENT gets its INDEX (and TOP for non-scopal composition) from the functor
 - Semantic operators where the result SEMENT gets its INDEX (and TOP for non-scopal composition) from the argument
- `op_non_scopal_functor_hook` – *eat + cake = eats cake*
- `op_non_scopal_argument_hook` – *tasty + cookie = tasty cookie*
- `op_scopal_functor_index` – *believes + it's Tuesday = believes it's Tuesday*
- `op_scopal_argument_index` – *probably + sleeps = probably sleeps*

Setting the SLOTS

- In Copestake et. al, 2001, the SLOTS list of the result comes from both SEMENTs
 - because the SLOTS are named for their syntactic function, conflicts where a slot from the functor and a slot from the argument have the same name do not occur (?)
- In my case, I name the slots after the bleached argument names that predications have in an MRS (ARG0, ARG1, etc.)
- This means if I take slots from both there are often conflicts
 - e.g. plugging `_find_v_1.ARG2` with `_box_n_of.ARG0`
 - both `_find_v_1` and `_box_n_of` have an ARG1, so if I propagate both for the new slots list I won't be able to distinguish between them just based on the name of the slot

Setting the SLOTS

- To handle this, I propagate the slots from the SEMENT that contributes the HOOK/INDEX information
 - this does mean that I will be unable to plug any remaining slots in the other SEMENT
 - ideally being clever about the order of composition can prevent getting stuck unable to plug a slot that was lost due to this decision
- `op_non_scopal_functor_hook_slots` – *eat* + *cake* = *eats cake*
- `op_non_scopal_argument_hook_slots` – *tasty* + *cookie* = *tasty cookie*
- `op_scopal_functor_index_slots` – *believes* + *it's Tuesday* = *believes it's Tuesday*
- `op_scopal_argument_index_slots` – *probably* + *sleeps* = *probably sleeps*

Full inventory of Fundamental Semantic Operations

- `op_create_base_SEMENT`
- `op_create_CARG_SEMENT`
- `op_non_scopal_functor_hook_slots`
- `op_non_scopal_argument_hook_slots`
- `op_scopal_functor_index_slots`
- `op_scopal_argument_index_slots`
- `op_scopal_quantifier`

Semantic Composition Library

Semantic Composition Library

- Library of functions that are named roughly for the syntactic/semantic construction they put together
 - e.g. `adjectival_modifier`— plugs the ARG1 of the functor `SEMENT` with the INDEX of the given nominal `SEMENT` and the result is a `SEMENT` where the adjective modifies the noun
- Each function in this library calls whatever sequence of Fundamental Semantic Operations it needs to to arrive at the result
 - e.g. `appositive` takes two arguments, `first_nominal_SEMENT` and `second_nominal_SEMENT` and does the following:
 1. Check if both given `SEMENT`s are quantified; quantify with an abstract quantifier if not
 2. Introduce a `SEMENT` with an EP labeled `appos`
 3. Plug `appos.ARG2` with `second_nominal_SEMENT.INDEX`
 4. Plug `appos.ARG1` with `first_nominal_SEMENT.INDEX`

Semantic Composition Library

- Users who wish to generate English text using my system should make use of the functions in this library
- The names and documentation of the functions should be transparent/accessible enough for users to choose the right ones
- Users therefore do not need to know the intimate details of MRS composition
 - e.g. knowing when composition is scopal vs. non scopal, knowing when “unseen” predications need to be introduced

Single Predicate Functions

- adjective
- comparative_adjective
- conjunction
- determiner
- noun
- number
- preposition
- pronoun
- quantifier
- verb
- name
- non_unique_name
- unknown_adjective
- unknown_noun
- unknown_verb
- manual_synopsis

Compositional Constructions that I'm happy with

- `adjectival_modifier`
- `appositive`
- `copula`
- `date`
- `compound_location`
- `compound_noun`
- `number_with_unit`
- `object_of_noun`
- `object_of_preposition`
- `cardinal_modifier`
- `ordinal_modifier`
- `passive_participle_modifier`
- `possessive`
- `prepositional_modifier`
- `present_participle_modifier`
- `quantify`
- `un_prefix`

Composition Functions I'm not happy with

- ARG1_relative_clause
- ARG2_relative_clause
- coordination
- intransitive_verb_sentence
- negation
- prepositional_relationship
- relative_direction
- transitive_verb_sentence

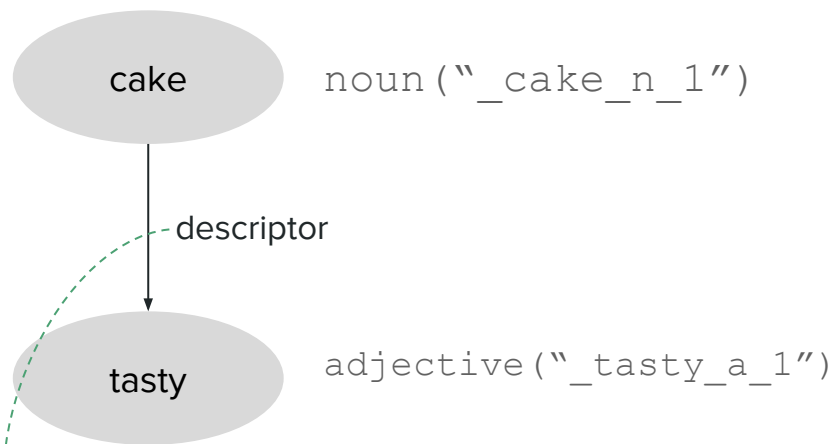
Summary

- The SCL (and the Fundamental Semantic Operations it calls) provides functions for building MRS structures “from scratch”
- The functions take semantic information (i.e. predicate labels or SEMENTs) as input and produce SEMENTs as output
- The remainder of my work is the tooling I have built around this to enable generating accurate MRSes that reflect the information in directed graphs
- But using my additional tooling is not required for producing SEMENTs with the SCL, and someone could use the SCL within a different architecture

The Lexicon

The Lexicon

- When trying to generate an MRS for a given graph, the central idea is that there is some MRS structure that will be generated for each node, and each edge will specify how to relate the parent and child nodes connected to that edge
- In order to do this, there must be a mapping from each unique node/edge type in a dataset to some function (or combination of functions) in the SCL



```
[ TOP: h0  
  INDEX: x1  
  RELS: [ _cake_n_1 LBL: h0 ARG0: x1 ]  
  EQS: <>  
  HCONS: <> ]
```

```
[ TOP: h2  
  INDEX: e3  
  RELS: [ _tasty_a_1 LBL: h0 ARG0: e3 ARG1: x4 ]  
  EQS: <>  
  HCONS: <> ]
```

`adjectival_modifier(child, parent)`

```
[ TOP: h0  
  INDEX: x1  
  RELS: [ _cake_n_1 LBL: h0 ARG0: x1 ],  
        [ _tasty_a_1 LBL: h0 ARG0: e3 ARG1: x4 ]  
  EQS: < h0 eq h1, x1 eq x4 >  
  HCONS: <> ]
```


Data-to-MRS Algorithm

Data-to-MRS Algorithm

- Recursive depth-first graph traversal algorithm
- Converts each node to a SEMENT using the functions specified in the lexicon for that node
- When returning back up the call stack composes a result SEMENT using the parent and child nodes for each edge using the functions specified in the lexicon for that edge

DEMO

